

Fall 2026 Semester Project Assignment:

SDI 4213-980: DevOps – CI/CD

Title: DevOps Pipeline for a Containerized Web Application Course

Project Overview

In this semester-long group project, your team will build, test, containerize, secure, deploy, and document a small web application or API using modern DevOps practices. The purpose of the project is not to build the most complex application possible. The purpose is to demonstrate that your team can use a professional software delivery workflow from development through deployment and operations.

The project will be completed in stages throughout the semester. Each stage will connect directly to the weekly course topics, including Git and GitHub collaboration, automated testing, continuous integration, Docker, Docker Compose, DevSecOps, cloud deployment, Infrastructure as Code, Kubernetes, observability, rollback planning, and operations documentation.

By the end of the semester, each team will demonstrate a complete end-to-end pipeline that can build, test, scan, package, deploy, and operate the application.

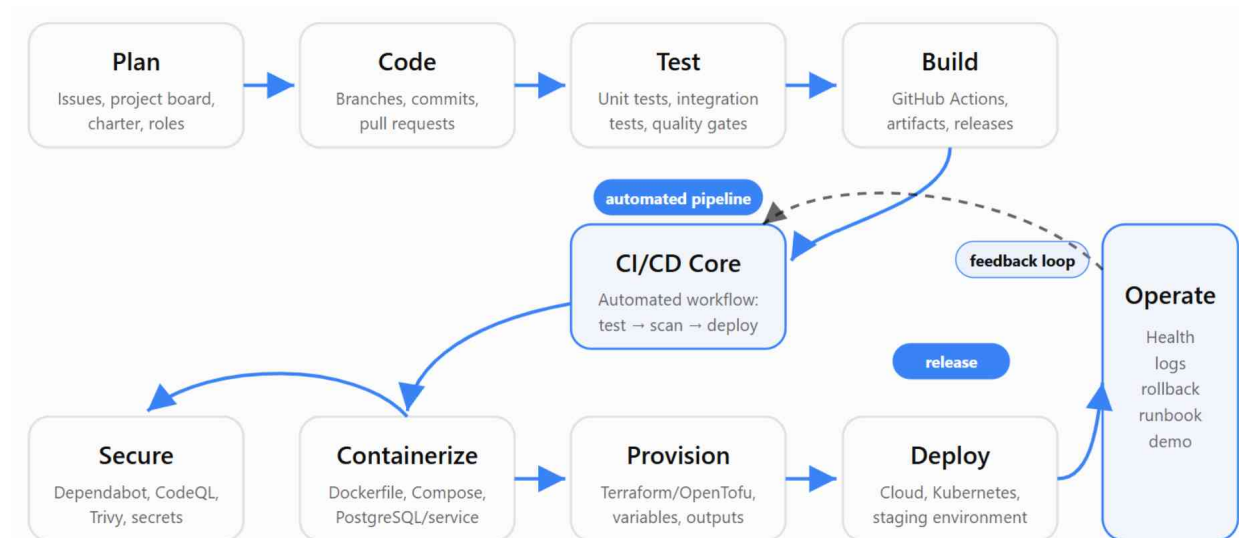


Figure: A flowchart illustrating a software development process. It starts with planning issues, project board, charter, and roles. Next is coding branches, commits, and pull requests. Testing involves unit tests, integration tests, and quality gates. Building creates GitHub actions, artifacts, and releases. CI/CD core handles automated pipelines with testing, scanning, and deployment. Feedback loop connects to operate, which manages logs, rollback, runbook, and demo, then releases.

Recommended Project Options

Each team will choose or be assigned a small application. The application should be simple enough to complete during the semester but realistic enough to support testing, deployment, and operations.

Recommended options include:

1. **Inventory Tracker**
A simple application for tracking items, quantities, categories, and status.
2. **Help Desk Ticket System**
A simple application where users submit tickets and administrators update ticket status.
3. **Task Tracker**
A simple application where users create, update, complete, and delete tasks.
4. **Course Resource Tracker**
A simple application for organizing course links, readings, tools, and assignments.
5. **Simple API with Dashboard**
A backend API with a lightweight frontend or documentation page.

Teams may propose a different project, but it must be approved by the instructor.

Suggested Tools

The project is designed so that required tools can be used at no cost.

Category	Required or Recommended Tool
Code editor	VS Code
Version control	Git
Repository hosting	GitHub
Project management	GitHub Issues and GitHub Projects
CI/CD	GitHub Actions
Application framework	FastAPI, Flask, Express, Spring Boot, or another approved framework
Testing	Pytest, Jest, JUnit, or another approved testing framework
Containerization	Docker
Multi-service local environment	Docker Compose
Security scanning	Dependabot, CodeQL, Trivy, Docker Scout, or approved equivalent

Infrastructure as Code	OpenTofu or Terraform
Kubernetes	Kubernetes with kind or Minikube
Cloud or hosted deployment	Render, Azure App Service, Google Cloud Run, AWS, or instructor-approved alternative
Documentation	Markdown, README files, Mermaid diagrams, screenshots, and/or diagrams.net

Students should not purchase paid software, select paid cloud service tiers, or enter payment information for cloud services unless explicitly approved by the instructor.

Final Project Requirements

Each team's final submission must include the following:

Requirement	Minimum Expectation
Working application	A functional web application or API
GitHub repository	Organized repository with meaningful commit history
Collaboration workflow	Issues, branches, pull requests, and code reviews
Automated testing	At least five automated tests
Continuous integration	GitHub Actions workflow that runs tests automatically
Build automation	Automated build step and release/tagging process
Docker support	Working Dockerfile
Docker Compose support	Multi-service local environment, preferably with a database
Security practices	At least two pipeline or application security controls
Deployment	Application deployed to an approved cloud, hosted, or local deployment environment
Infrastructure as Code	OpenTofu or Terraform configuration for at least one resource or deployment component
Kubernetes deployment	Kubernetes manifests that run locally using kind or Minikube
Observability	Health endpoint, useful logs, and deployment validation steps
Operations documentation	Runbook with deployment, rollback, troubleshooting, and security notes

Final presentation Live or recorded demonstration of the DevOps workflow

Weekly Milestones

Week 1: Project Charter and Repository Setup

Your team will select a project idea, form roles, and create the initial GitHub repository.

Deliverables:

- Team name and team members
- Project title
- Short project description
- Target users
- Planned application features
- Initial technology stack
- GitHub repository link
- Initial README file

Suggested project charter format:

Project Name:

Team Members:

Application Purpose:

Target Users:

Primary Features:

Technology Stack:

Repository Link:

Week 2: GitHub Workflow and Collaboration Setup

Your team will configure the repository for collaborative development.

Required items:

- Branching strategy
- GitHub Issues
- GitHub Project board or equivalent task board
- Pull request template
- Contribution guidelines

- At least one feature branch
- At least one pull request

Deliverable:

Submit a link to the repository showing the GitHub workflow setup.

Week 3: Application Skeleton and Initial Testing

Your team will create the first working version of the application.

Minimum requirements:

- Basic application structure
- At least one working route, page, or API endpoint
- At least three automated tests
- Instructions for running the application locally

Example API endpoints:

GET /health

GET /items

POST /items

GET /items/{id}

Deliverable:

Submit a working application skeleton with tests.

Week 4: Continuous Integration with GitHub Actions

Your team will add a CI workflow using GitHub Actions.

Minimum requirements:

- Workflow runs on pull requests
- Workflow checks out the code
- Workflow installs dependencies
- Workflow runs automated tests
- Failed tests prevent successful completion of the workflow

Deliverable:

Submit a pull request showing that the CI workflow runs automatically.

Week 5: Build Automation and Release Practices

Your team will add build and release practices to the project.

Minimum requirements:

- Automated build step
- Version number
- Release tag, such as v0.1.0
- Release notes or changelog entry
- Build artifact, if applicable

Deliverable:

Submit a tagged release and describe what changed in that release.

Week 6: Docker Containerization

Your team will containerize the application.

Minimum requirements:

- Working Dockerfile
- Application runs locally in a container
- Documented build and run commands
- Environment variables handled appropriately

Example commands:

```
docker build -t team-app .
```

```
docker run -p 8000:8000 team-app
```

Deliverable:

Submit the Dockerfile and README instructions for building and running the container.

Week 7: Docker Compose and Multi-Service Environment

Your team will use Docker Compose to run the application with at least one supporting service.

Minimum requirements:

- docker-compose.yml
- Application service
- Database or supporting service

- Environment variables
- Network configuration
- Volume configuration, if using persistent data

Example command:

`docker compose up`

Deliverable:

Submit a working Docker Compose setup and local run instructions.

Week 8: Midterm Project Checkpoint

The midterm checkpoint is a practical integration milestone.

Required by Week 8:

- GitHub repository
- GitHub Issues and project board
- Pull request workflow
- Working application
- Automated tests
- GitHub Actions CI workflow
- Dockerfile
- Docker Compose configuration
- README with setup instructions

Deliverable:

Submit the repository link and a short checkpoint summary explaining what is working and what still needs improvement.

Week 9: DevSecOps and Pipeline Security

Your team will add security practices to the project.

Minimum requirement:

Add at least two of the following:

- GitHub Dependabot
- GitHub CodeQL
- Trivy container image scanning
- Docker Scout

- Secret scanning
- .env.example file
- Removal of hard-coded secrets
- Least-privilege GitHub Actions permissions
- Dependency vulnerability review

Deliverable:

Submit evidence of security scanning or security controls and briefly explain what your team added.

Week 10: Continuous Deployment to Staging

Your team will create a staging deployment workflow.

Minimum requirements:

- Deployment workflow triggered after merge to main
- Staging environment or staging-like deployment target
- Deployment URL or access instructions
- Basic deployment verification step
- Manual approval or environment protection, if available

Deliverable:

Submit evidence that the application deploys to a staging environment.

Week 11: Cloud or Hosted Deployment

Your team will deploy the containerized application to an approved cloud or hosted environment.

Approved options may include:

- Render
- Azure App Service
- Google Cloud Run
- AWS Elastic Beanstalk, ECS, or App Runner
- Instructor-approved alternative

Minimum requirements:

- Application is accessible to the instructor
- Deployment process is documented
- Required environment variables are documented

- No paid service tier is used unless approved

Deliverable:

Submit the deployed application URL or access instructions.

Week 12: Infrastructure as Code

Your team will use OpenTofu or Terraform to define at least one infrastructure or deployment-related resource.

Minimum requirements:

- Provider configuration
- At least one resource
- Variables
- Outputs
- Documented init, plan, and apply process

Acceptable options:

- Provision a simple cloud resource
- Manage a local Docker resource
- Manage a GitHub repository setting
- Configure another instructor-approved resource

Deliverable:

Submit the IaC files and evidence that your configuration was planned and/or applied successfully.

Week 13: Kubernetes Deployment

Your team will deploy the application using Kubernetes locally.

Minimum requirements:

- deployment.yaml
- service.yaml
- ConfigMap or environment configuration
- Secret example or documented secret-handling approach
- Replicas configured
- Health check or readiness/liveness probe, if appropriate
- Deployment works in kind or Minikube

Example commands:

```
kubectl apply -f k8s/
```

```
kubectl get pods
```

```
kubectl get services
```

Deliverable:

Submit Kubernetes manifests and evidence that the application runs locally in Kubernetes.

Week 14: Observability, Operations, and Rollback

Your team will prepare the project for operation and maintenance.

Minimum requirements:

- /health endpoint or equivalent health check
- Useful application logs
- Deployment validation steps
- Rollback plan
- Troubleshooting guide
- Architecture diagram
- Pipeline diagram
- Operations runbook

Suggested runbook sections:

1. System overview
2. How to run locally
3. How to deploy
4. How to verify deployment
5. How to view logs
6. How to rollback
7. Known issues
8. Security considerations

Deliverable:

Submit the operations runbook and diagrams.

Week 15: Final Project Completion

Your team will use this week to complete, troubleshoot, polish, and prepare the final demonstration.

Required work:

- Finish incomplete technical components
- Clean repository structure
- Update README
- Confirm all workflows run correctly
- Confirm deployment is accessible
- Finalize diagrams
- Finalize runbook
- Prepare presentation or demo script

Deliverable:

Submit a final readiness checklist.

Week 16: Final Presentation and Demonstration

Each team will present and demonstrate the completed project.

The final demo should show:

1. GitHub repository organization
2. Issues, branches, pull requests, and code reviews
3. Automated tests
4. CI workflow
5. Docker build
6. Docker Compose local environment
7. Security scan or security controls
8. Deployment workflow
9. Deployed application
10. Infrastructure as Code component
11. Kubernetes deployment
12. Health check, logs, rollback plan, and operations documentation

Deliverable:

Final presentation, final repository link, deployment link, and completed documentation.

Final Submission Package

Each team must submit:

1. GitHub repository link
2. Deployed application link or access instructions
3. README file
4. Dockerfile
5. Docker Compose file
6. GitHub Actions workflows
7. Security scanning evidence
8. OpenTofu or Terraform files
9. Kubernetes manifests
10. Architecture diagram
11. Pipeline diagram
12. Operations runbook
13. Final presentation slides or demo outline
14. Individual contribution statement from each team member

Suggested Repository Structure

project-name/

README.md

CONTRIBUTING.md

CHANGELOG.md

Dockerfile

docker-compose.yml

.github/

workflows/

ci.yml

deploy.yml

app/

...

tests/

...

infra/

main.tf

variables.tf

outputs.tf

k8s/

deployment.yaml

service.yaml

configmap.yaml

secret-example.yaml

docs/

architecture.md

pipeline.md

runbook.md

security.md

Teams may use a different structure if it is clearly documented.

Grading Rubric For Final Presentation

Category	Weight
Application functionality	10%
GitHub workflow and collaboration	10%
Automated testing	10%
Continuous integration pipeline	10%
Docker and Docker Compose	10%
Security practices	10%
Cloud or hosted deployment	10%
Infrastructure as Code	10%
Kubernetes deployment	10%
Documentation, runbook, and final presentation	10%

Individual Contribution

Although the project is completed in teams, each student must submit a brief individual contribution statement at the end of the semester.

The statement should include:

- Your primary responsibilities
- Specific files, features, workflows, or documentation you contributed
- Pull requests or issues you worked on
- What you learned
- What you would improve with more time

Course Cost and Cloud Usage Policy

The project is designed to be completed using free tools and resources. Students should not purchase software, select paid cloud tiers, or enter payment information unless explicitly approved by the instructor.

If a cloud platform requests payment information or requires a paid tier, students should stop and ask the instructor for an alternative. Local deployment using Docker Compose, kind, Minikube, or another approved no-cost option may be used when appropriate.

Academic Integrity and Collaboration

Students are encouraged to use documentation, tutorials, official examples, and AI-assisted tools appropriately. However, each team must understand, explain, and demonstrate the work they submit.

Teams must not submit copied projects without meaningful modification, documentation, and understanding. All outside code, templates, tutorials, or generated code should be cited or acknowledged in the repository documentation.

Project Goal

By completing this project, students will demonstrate the ability to move software through a realistic DevOps lifecycle:

Plan → Code → Test → Build → Containerize → Scan → Deploy → Monitor → Operate → Improve

The final project should show not only that the application works, but that the team can manage the full process required to deliver and maintain software in a modern development environment.